ual: creative computing institute

CODING FOR DIGITAL MEDIA · WEEK 4

Cut and Paste Audio

Sound is a list of numbers – so we can cut it, paste it and rearrange it in code

Louis McCallum · Autumn 2026

THIS WEEK

Four questions about audio

1. What is sound – and how does a computer record it?
2. What is actually inside an audio file?
3. Who cut and pasted sound before computers?
4. How do we cut, paste and edit audio in Python?

THE ROUTE

From vibrations to a sound collage

1

Sound

Vibrations, microphones, voltage

Q1

2

Digital audio

Sampling – audio is a list of numbers

Q1 · Q2

3

Cut & paste

Tape, samplers, hip-hop, plunderphonics

Q3

4

Cutting

Slicing lists – in seconds and in beats

Q4

5

Pasting & editing

Overwrite, mix, volume, fades, reverse, speed

Q4

Each part builds on the last: **sound** becomes **numbers**; composers learned to **cut and paste** recorded sound; we do the same by **indexing lists**. Then you make a **collage**.

● TODAY'S CODE

The techniques and their files

Technique	File	Part
Load, play and draw a waveform	week4-waveform.py	2 · Digital audio
Bars and beats	week4-bars.py	4 · Cutting
Overwrite part of a sound	week4-paste.py	5 · Pasting
Mix two sounds	week4-mix.py	5 · Pasting
Volume	week4-quieter.py	5 · Editing
Fade in	week4-fadein.py	5 · Editing
Reverse	week4-reverse.py	5 · Editing
Speed and pitch	week4-faster.py · week4-slower.py	5 · Editing

librosa loads audio into Python; **Dorothy** plays it and draws it, as in previous weeks.

Part 1

Sound

Before any code: what is it we're actually recording?

Vibrations · air pressure · microphones · voltage · converting to numbers

What is sound?



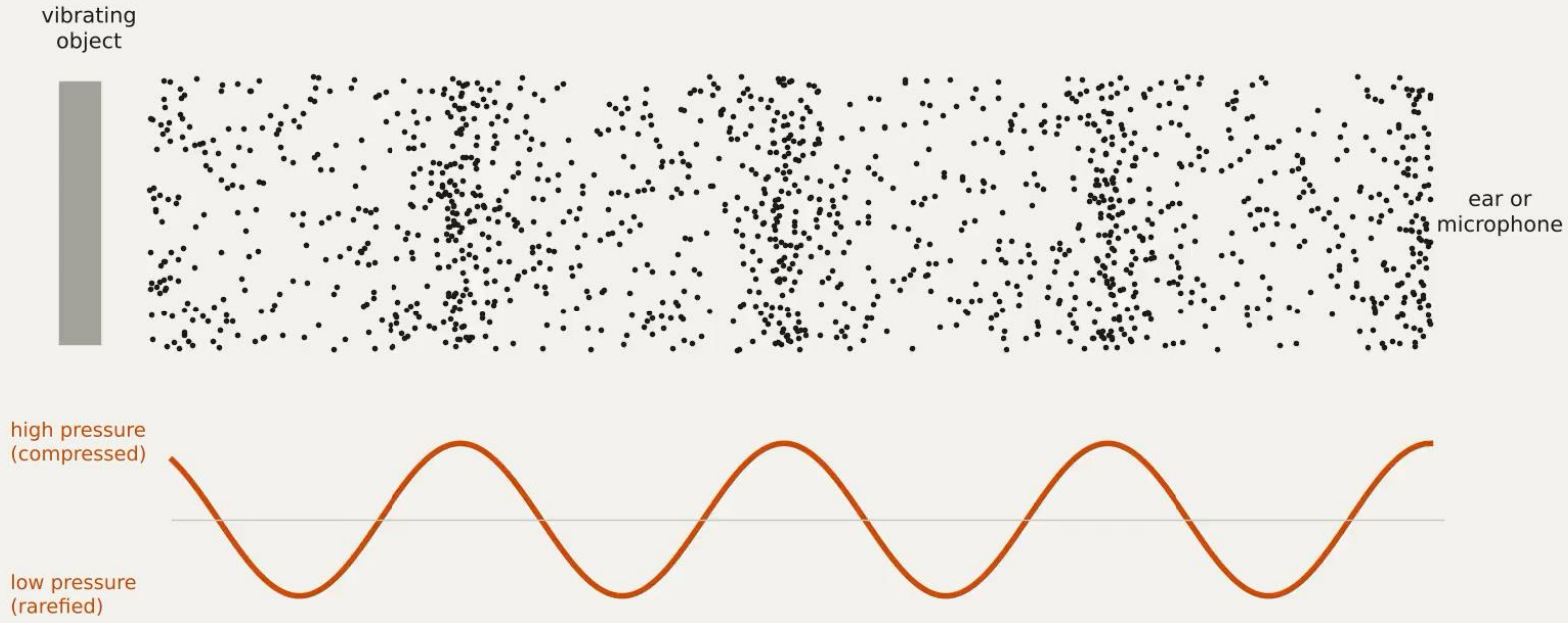
Sound is **vibrations**.

It can travel through **air** – or other matter: water, wood, metal, the ground.

Something vibrates, and pushes on whatever is next to it.

▶ Watch: Questacon tuning fork demo

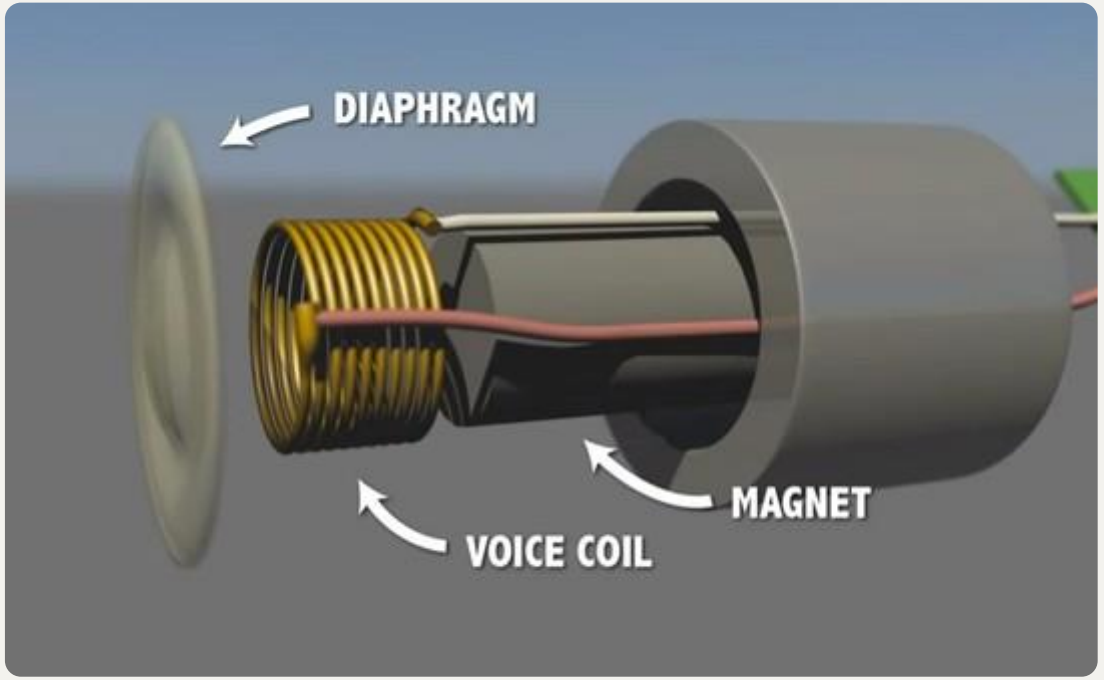
Air molecules compressing and rarefying



The molecules don't travel across the room – the **pattern of pressure** does. That pattern is the wave we draw.

▶ Watch: sound waves in air

Recording sound: vibrations become voltage



The **diaphragm** is pushed back and forth by the changing air pressure.

It moves a **coil** past a **magnet**, which makes a small changing **voltage**.

A microphone is a **transducer**: it converts a signal from one form of energy into another.

▶ Watch: inside a dynamic microphone

From air to a file

VIBRATION

Air pressure goes up and down

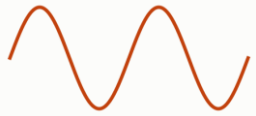
→

MICROPHONE

A transducer: pressure → voltage

→

VOLTAGE



A smooth, continuous signal

→

ADC

Analog-to-digital converter: measures the voltage thousands of times a second

→

NUMBERS



One number per measurement → SoundFile.wav

PLAYBACK Runs it backwards: numbers → **DAC** (digital-to-analog) → voltage → speaker → vibrations in the air.

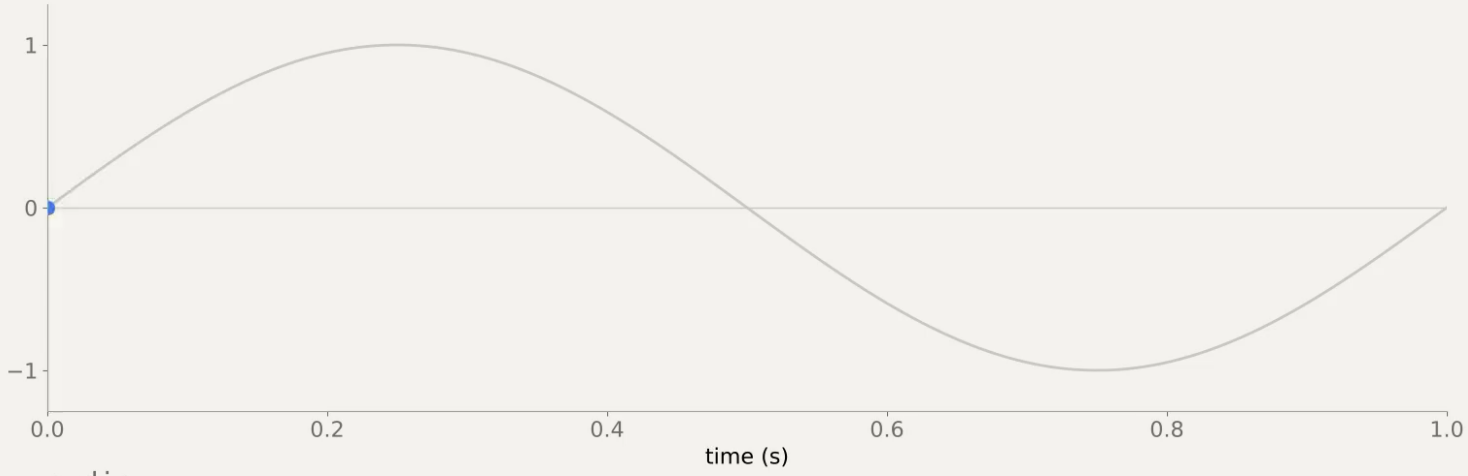
Part 2

Digital audio

A computer can’t store a continuous wave. It stores measurements – a list of numbers.

Sampling · sample rate · audio as a list · librosa · waveforms

Sampling: measuring at regular intervals

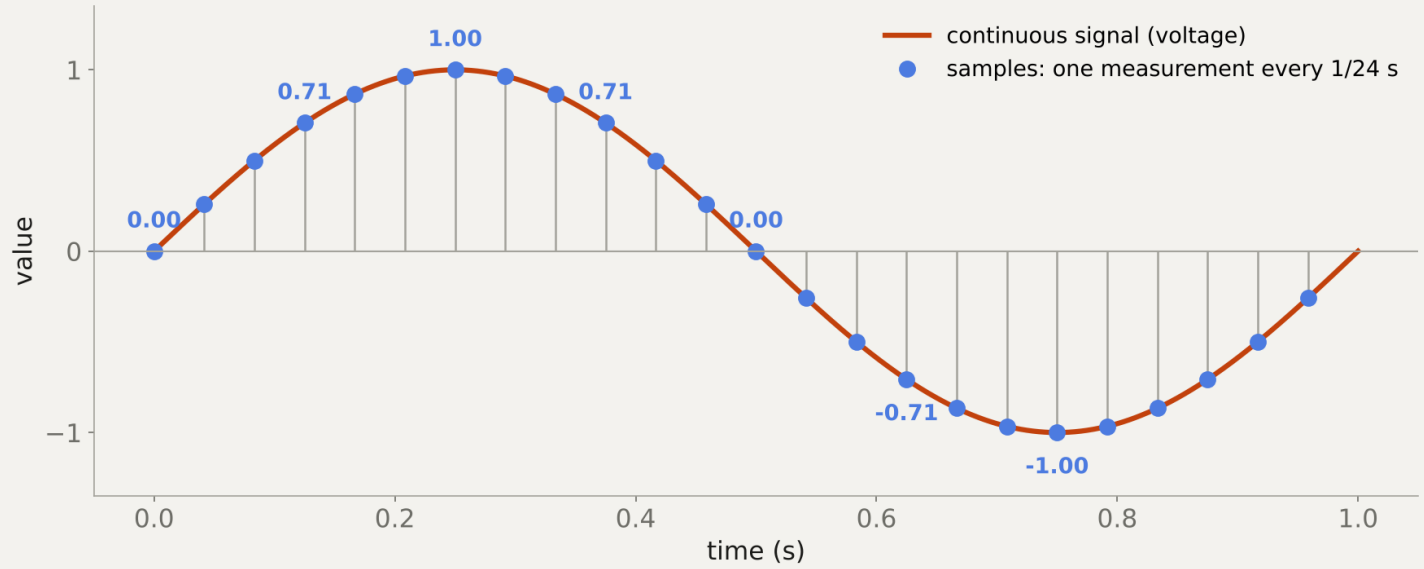



```
audio =

[0.00
```

Each measurement is a **sample**. Put them in order and you have a **list of numbers** that represents the sound.

Sample rate: measurements per second

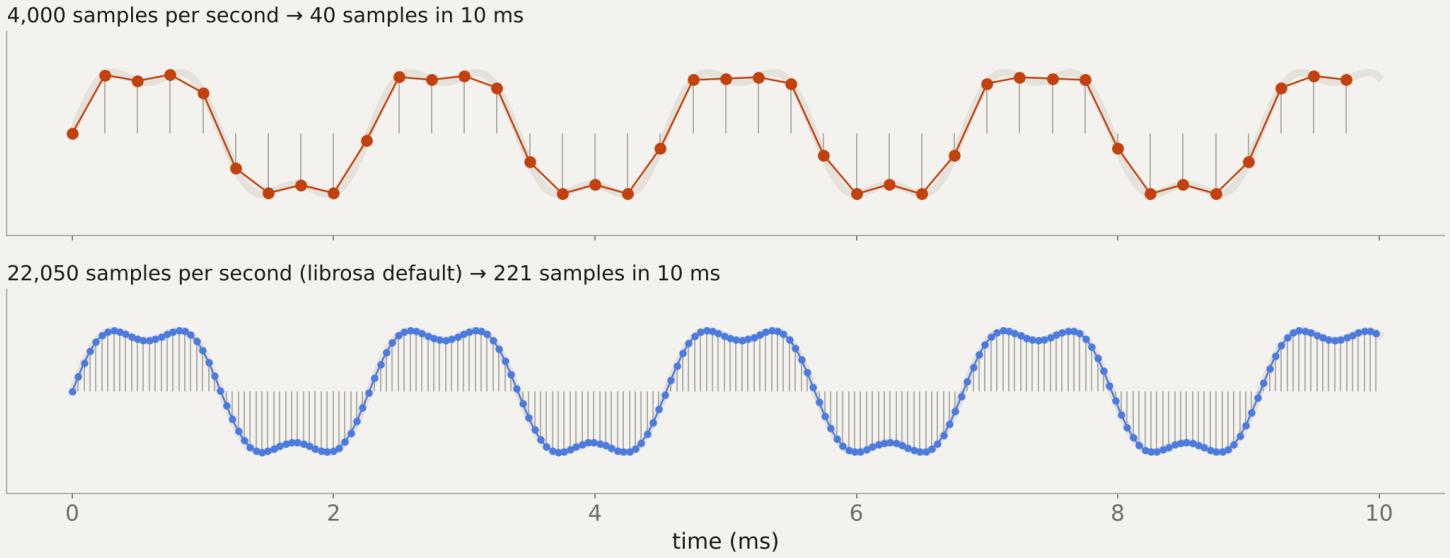


We can't represent continuous signals on a computer, so we **sample** them at regular intervals.

The number of samples per second is the **sample rate**, in hertz (Hz).

These numbers are what is in an **uncompressed** audio file, e.g. **.wav**.

More samples, more detail



CD AUDIO

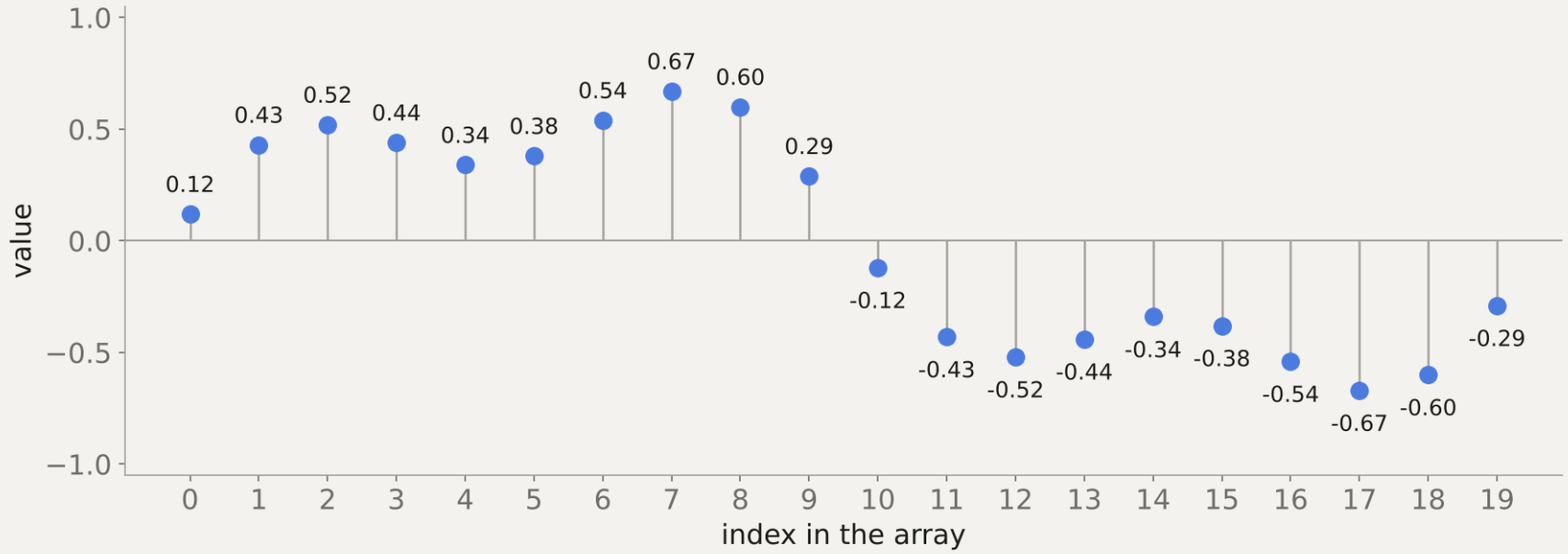
44,100 samples per second

LIBROSA DEFAULT

22,050 samples per second

The highest pitch a sample rate can capture is **half** of it: 44,100 Hz covers all of human hearing (about 20,000 Hz).

Audio files are just lists




```
audio = [0.12, 0.43, 0.52, 0.44, 0.34, 0.38, 0.54, 0.67, 0.60, 0.29, ...]
```

How precise is each number?

4-BIT (TEXTBOOK DIAGRAMS)

16 possible levels, 0–15. Easy to draw, sounds terrible.

16-BIT (CD, MOST .WAV FILES)

65,536 possible levels per sample.

LIBROSA

Decimal numbers (floats) between **−1.0** and **1.0**, whatever the file stored.

The **sample rate** is how often we measure. The **bit depth** is how finely we measure. Both decide how much data a recording takes.

How much data is that?

SAMPLE RATE		BIT DEPTH		STEREO		ONE MINUTE
44,100	×	**2 bytes**	×	**2**	=	**10.6 MB**
samples per second		16 bits per sample		channels, left and right		uncompressed, CD quality

That's why **MP3** and **AAC** exist: they throw away detail you're unlikely to hear. For editing, we want every sample – so we use **.wav**.

librosa: loading audio in Python

```
#week4-waveform.py
def setup(self):
    audio, sr = librosa.load("audio/drums.wav")
    num_samples = len(audio)
    print(f"{num_samples} samples at {sr}Hz lasts for {(num_samples / sr):.2f} seconds")
    dot.music.start_sample_stream(audio, sr=sr)
```

librosa.load()

Gives back **two** things: the samples (as an array) and the sample rate.

22,050

librosa's sample rate unless you specify another one.

start_sample_stream

Dorothy plays the samples out of the speakers.

How long is it?

len(audio)
99,849
samples

÷

sr
22,050
samples per second

=

DURATION
4.53 s
of drums

OUTPUT
99849 samples at 22050Hz lasts for 4.53 seconds

Number of samples ÷ sample rate = length in seconds. We'll use this sum all week.

Format strings: numbers inside text

```
print(f"{num_samples} samples at {sr}Hz lasts for {(num_samples / sr):.2f} seconds")
```

1

Put f in front of the opening "

2

Put any variable – or sum – inside { }

3

:.2f after a number: show 2 decimal places

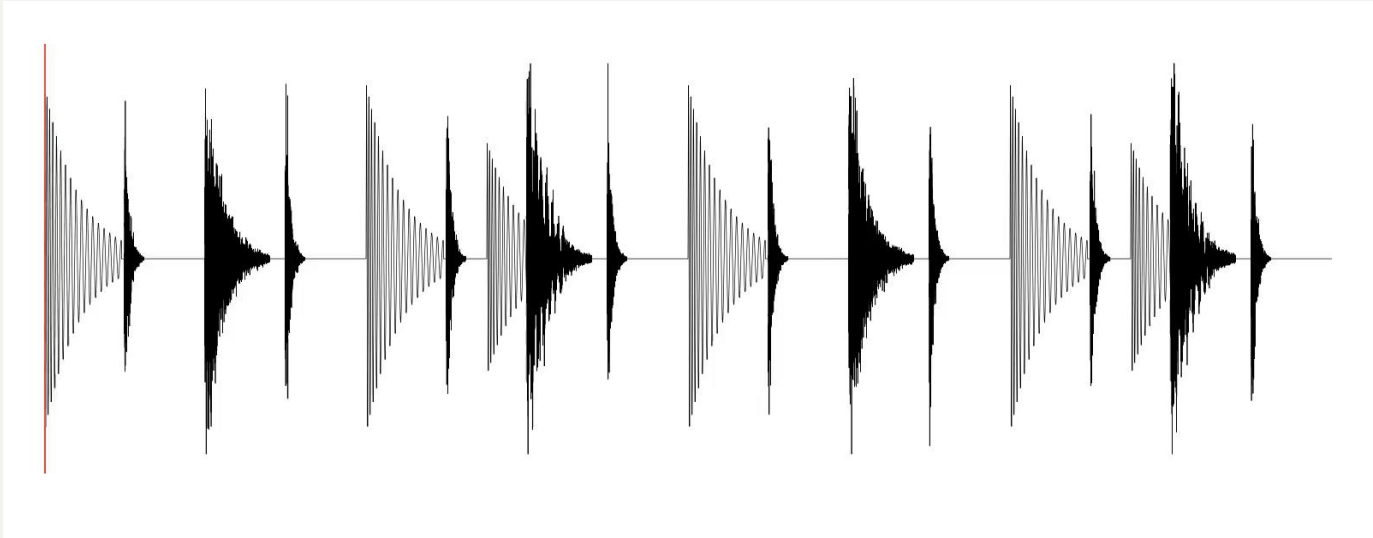
Without :.2f you'd see **4.528299319727891**.

Seeing the samples: drawing the waveform

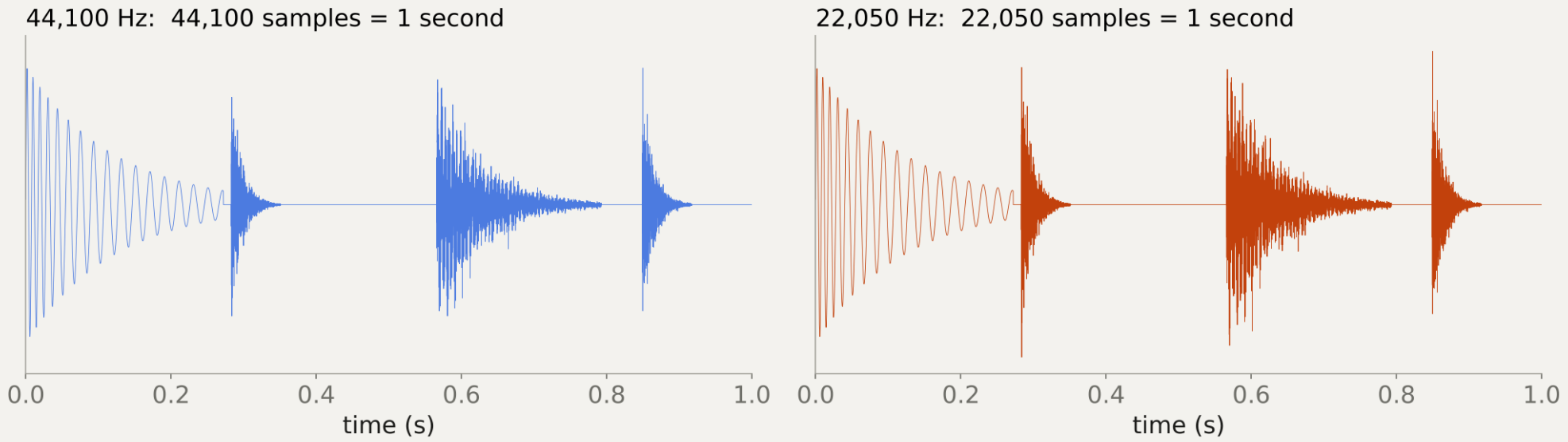
```
#week4-waveform.py
def draw(self):
    dot.background((255,255,255))
    dot.draw_waveform(col=dot.black,
                      with_playhead=True)
```

Pass a **colour**, and a **boolean**: draw a playhead or not.

Click the picture to play it – with sound.

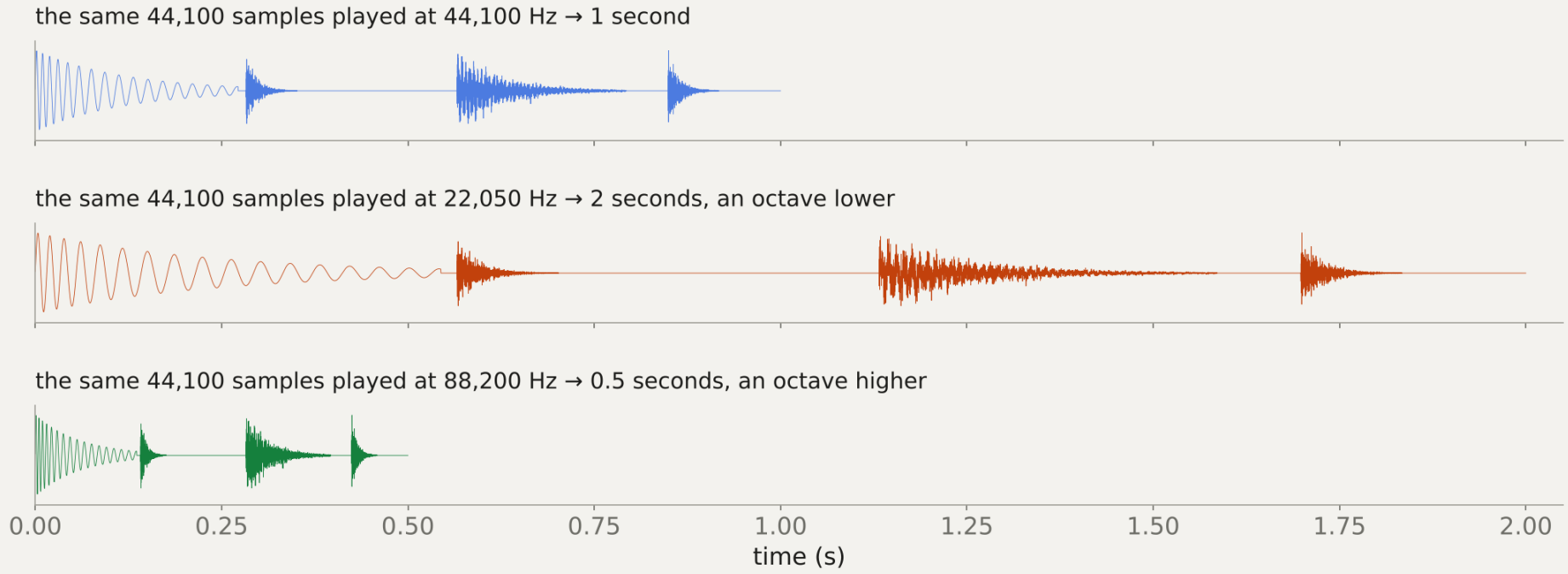


Same length, different numbers of samples



Depending on the **sample rate**, you need different numbers of samples to make the same length of audio. Both of these are **1 second** long.

Same samples, different lengths



Turn it round: the **same** samples played at a different rate last a different time – and change **pitch**. We'll use this in Part 5.

Part 3

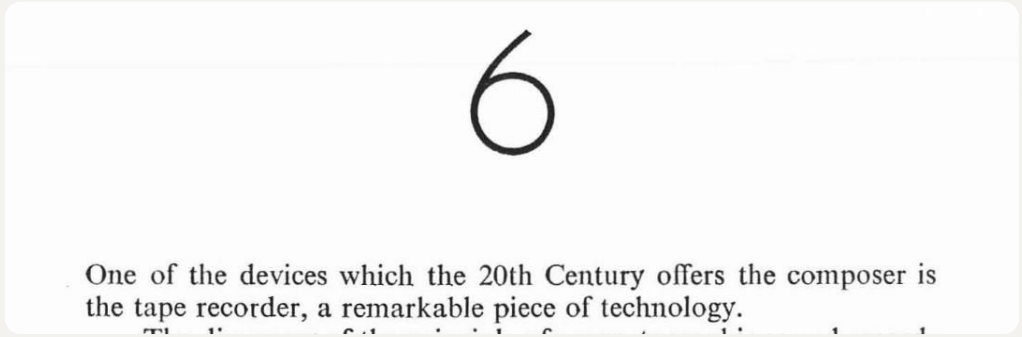
A history of cut and paste

Long before computers, composers and DJs were cutting up recorded sound to make something new.

Tape · musique concrète · the Radiophonic Workshop · samplers · hip-hop · plunderphonics · mashups

Cutting tape

Composers have used the physical cut and paste of tape to make new compositions for most of the 20th century.



6

One of the devices which the 20th Century offers the composer is the tape recorder, a remarkable piece of technology.

Daphne Oram, *An Individual Note of Music, Sound and Electronics*

▶ Watch: cutting tape



Pierre Schaeffer: musique concrète



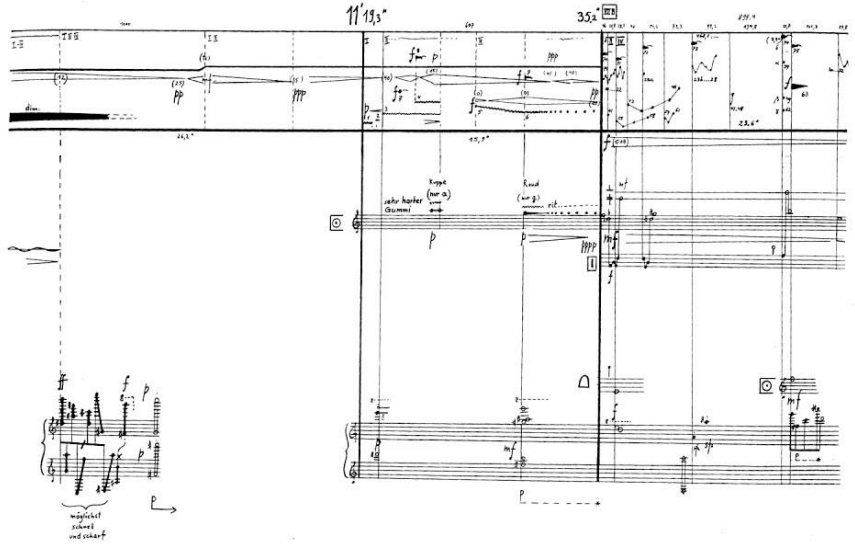
Often credited with pioneering *musique concrète* in 1940s France: **recorded sounds**, instead of musical notation, as raw material.

His work involved cutting, splicing and manipulating recordings – first on records, then on **magnetic tape**.

Étude aux chemins de fer (1948), made from train sounds, is considered one of the first pieces of musique concrète.

▶ **Watch the video**

Karlheinz Stockhausen

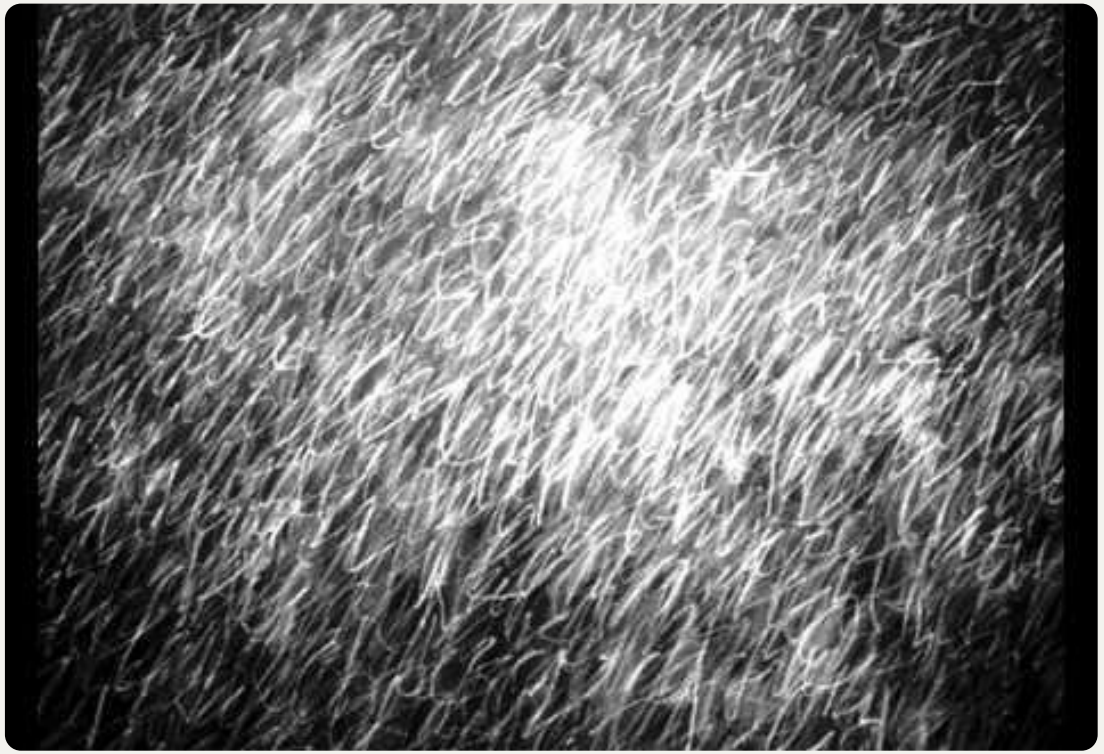


A leading figure in electronic music: his early works in the 1950s involved **manipulating tape**.

Pieces like *Gesang der Jünglinge* (1955–56) and *Kontakte* (1958–60) used tape techniques in combination with **electronic sounds**.

▶ Watch the video

John Cage: *Williams Mix*



More widely known for his **chance-based** and avant-garde works…

…his use of tape in pieces like *Williams Mix* (1952) was an early exploration of the potential of **splicing and editing** pre-recorded sounds.

▶ Watch the video

Daphne Oram



Co-founder of the **BBC Radiophonic Workshop**; one of the first composers in the UK to experiment with tape and sound manipulation.

Her work in the late 1950s and 1960s contributed to the development of electronic music.

Oramics: her technique of drawing sound waves onto film to control pitch and timbre, built on early tape practices.

▶ Watch the video

Delia Derbyshire



Also a member of the BBC Radiophonic Workshop.

Used **tape-splicing** techniques to create complex sound collages: layering **tape loops**, cutting and reassembling tape to achieve precise results.

Best known for realising the original *Doctor Who* theme (1963), built note by note from spliced tape.

▶ **Watch the video**

Into popular culture: the Mellotron



In the 1960s, the **Mellotron** was an early instrument that used **tape loops** to simulate orchestral sounds.

Not strictly a sampler in the modern sense, but replaying pre-recorded sounds from a keyboard was an important step towards the sampling culture of the 1970s and beyond.

Famously used by The Beatles (e.g. *Strawberry Fields Forever*) and King Crimson.

▶ **Watch: The Mellotron (1965)**

Hip-hop DJs: live sampling



In the late 1970s and early 1980s, hip-hop DJs such as **Kool Herc**, **Grandmaster Flash** and **Afrika Bambaataa** began to use turntables in innovative ways, manipulating vinyl records as a form of live “sampling”.

An early form of musical sampling that didn’t yet rely on digital tools, but used **physical media**.

▶ Watch the video

The Fairlight CMI



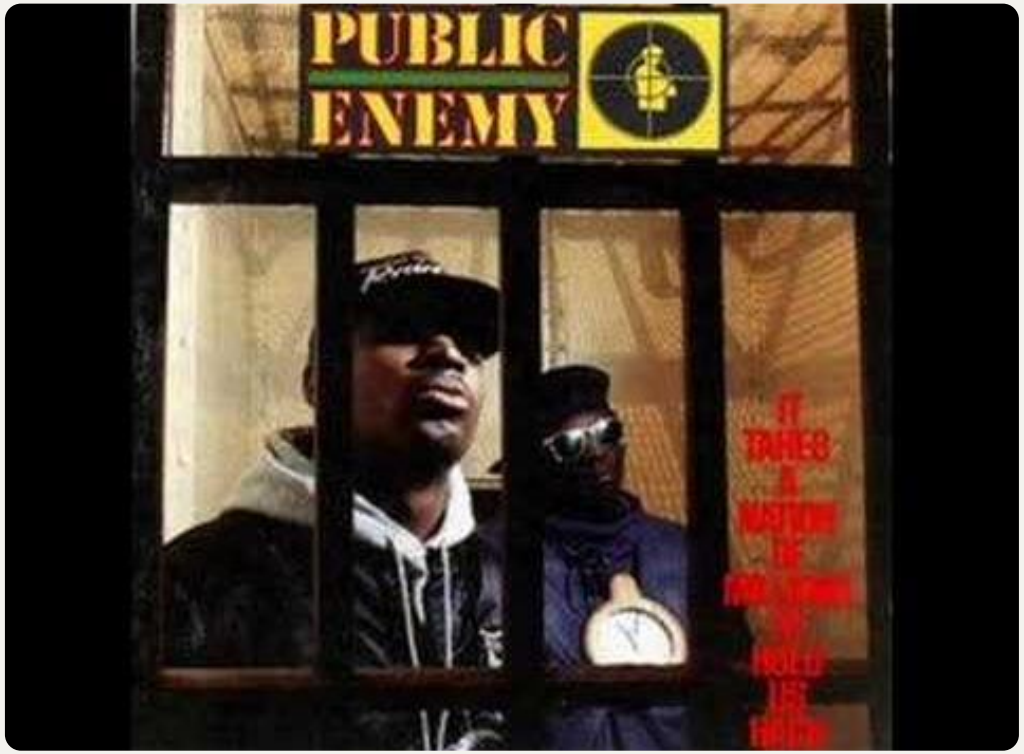
In the late 1970s the **Fairlight CMI** became one of the first commercially available **digital sampling** instruments.

Pioneers like Peter Gabriel, Kate Bush and Jean-Michel Jarre used it to sample real-world sounds and manipulate them digitally.

It shaped the sound of 1980s pop and experimental music: Gabriel's *Shock the Monkey*, Bush's *Hounds of Love*.

▶ **Watch: The Fairlight Story (Kate Bush at 9:25)**

Samplers and hip-hop production



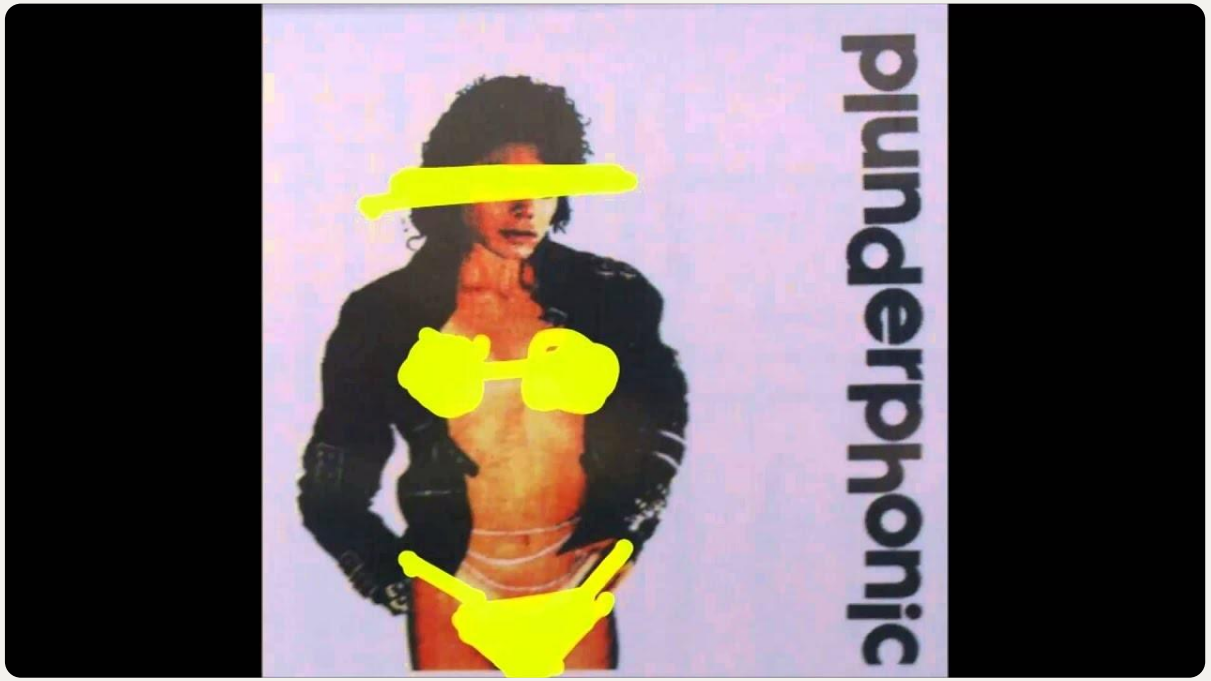
The **E-mu SP-1200** and **Akai MPC** samplers of the 1980s revolutionised hip-hop.

Producers like **Marley Marl**, **DJ Premier** and **The Bomb Squad** (who produced Public Enemy) pioneered sampling short sound bites from other records to build complex sound collages.

Public Enemy's *It Takes a Nation of Millions to Hold Us Back* (1988) is often cited as a landmark: hundreds of tiny fragments blended into a cohesive whole.

▶ **Watch the video**

John Oswald: Plunderphonics



Oswald's 1989 album *Plunderphonic* was a manifesto of sorts, built entirely from **fragments of existing popular music**.

In *Dab*, he transformed Michael Jackson's *Bad* by slowing it down and restructuring it – a warped, often surreal version of the original.

Quickly targeted by legal action from record companies over **copyright**, it laid the groundwork for plunderphonics as a distinct artistic practice.

▶ Watch the video

Mashup

More recent artists have layered (sometimes) **thousands of samples** together into cohesive compositions, often recombining popular music into new pieces.

DJ Shadow

The Avalanches

DJ Yoda

DJ Food

SPECIAL SHOUT DJ Food – *Raiding the 20th Century (Words & Music Expansion)* feat. Strictly Kev, Paul Morley & a Cast of Thousands…

▶ **Listen: Raiding the 20th Century on SoundCloud**

From razor blades to code

1948
Schaeffer
musique concrète

1952
Cage
Williams Mix

1950s
Stockhausen
tape + electronics

1958
Radiophonic Workshop
Oram, Derbyshire

1960s
Mellotron
tape-loop keyboard

1970s
Hip-hop DJs
live looping on vinyl

1979
Fairlight CMI
digital sampling

late 1980s
SP-1200 · MPC
Nation of Millions

1989
Oswald
Plunderphonic

2000s
Mashups
thousands of samples

PHYSICAL MEDIA

Tape, records, tape loops – cut with a razor blade, joined with sticky tape.

DIGITAL SAMPLES

From the Fairlight on: sound as **numbers in memory** – the same lists we use in Python.

Part 4

Cutting

An audio file is just a list. Cutting it is just indexing – more than one item at a time.

Lists recap · arrays · slicing · musical timing

Recap: variables and lists

Named variables replace actual values – for consistency, reuse and modularity.

```
bpm = 140
set_tempo(bpm)
```

A **List** stores a whole collection. Get or overwrite one item with **[index]**, starting at **0**.

```
#make a list
colour = [255,26,60]
#get an item
r = colour[0]
#update the list
colour[1] = 34
```

THIS WEEK To cut audio we need a **sequence of contiguous samples** – many items at once.

Our audio is a NumPy array

WHAT librosa GIVES YOU

A numpy.ndarray – a list of numbers built for maths.

INDEXING

Works **exactly like a List**: audio[0], audio[-1], len(audio).

THE DIFFERENCE

Arithmetic happens **item by item**: audio * 0.5 halves every sample. Part 5 uses this a lot.

We'll say "list" in these slides – just remember that it's an **array** when the maths matters.

Slicing: [start:end]



START

is **included**. Leave it out: from the beginning.

END

is **not** included. Leave it out: to the end.

LENGTH

end – start items: audio[2:5] has 3.

Cutting by seconds

```
audio, sr = librosa.load("audio/drums.wav")
#Get from the beginning to sr (this is the first second)
beginning = audio[:sr]
print(len(beginning))
#Get from the (end - sr) to the end (this is the last second)
end = audio[-sr:]
print(len(end))
#Get from sr to sr*3 (this is second 1-3)
mid = audio[sr:sr*3]
print(len(mid))
```

beginning

22050 samples = 1 s

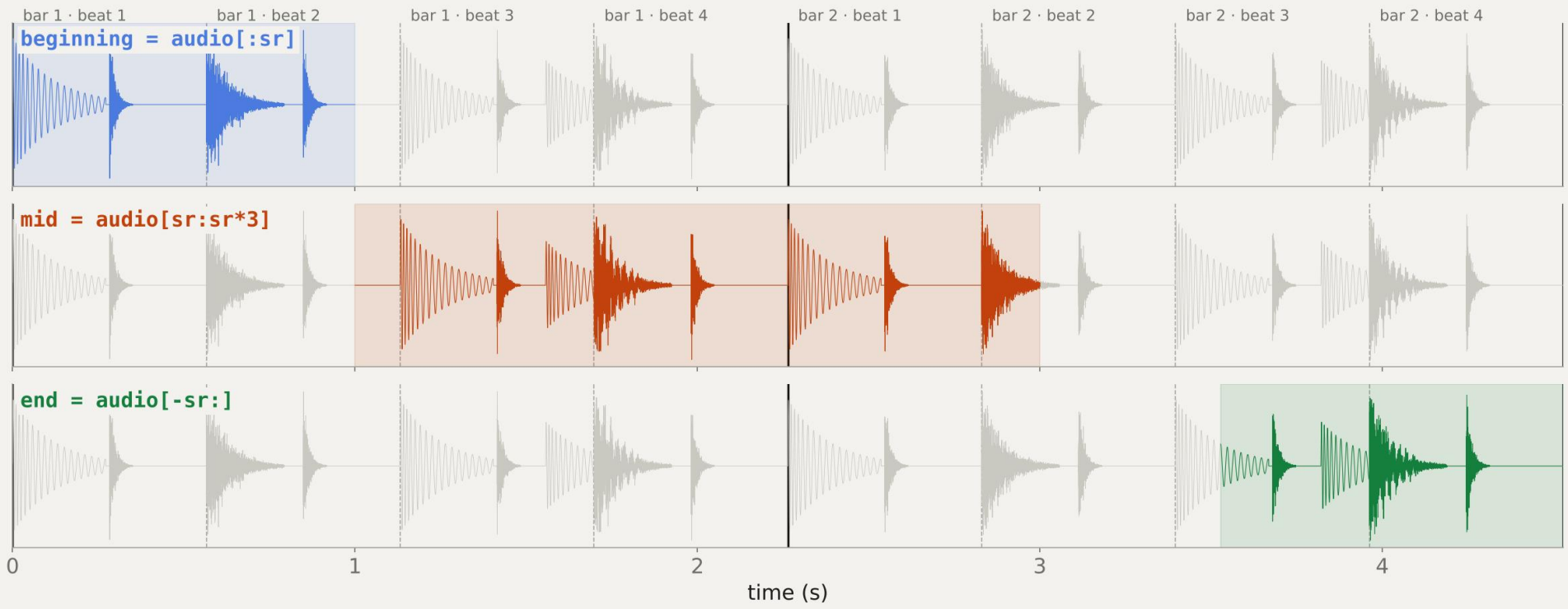
end

22050 samples = 1 s

mid

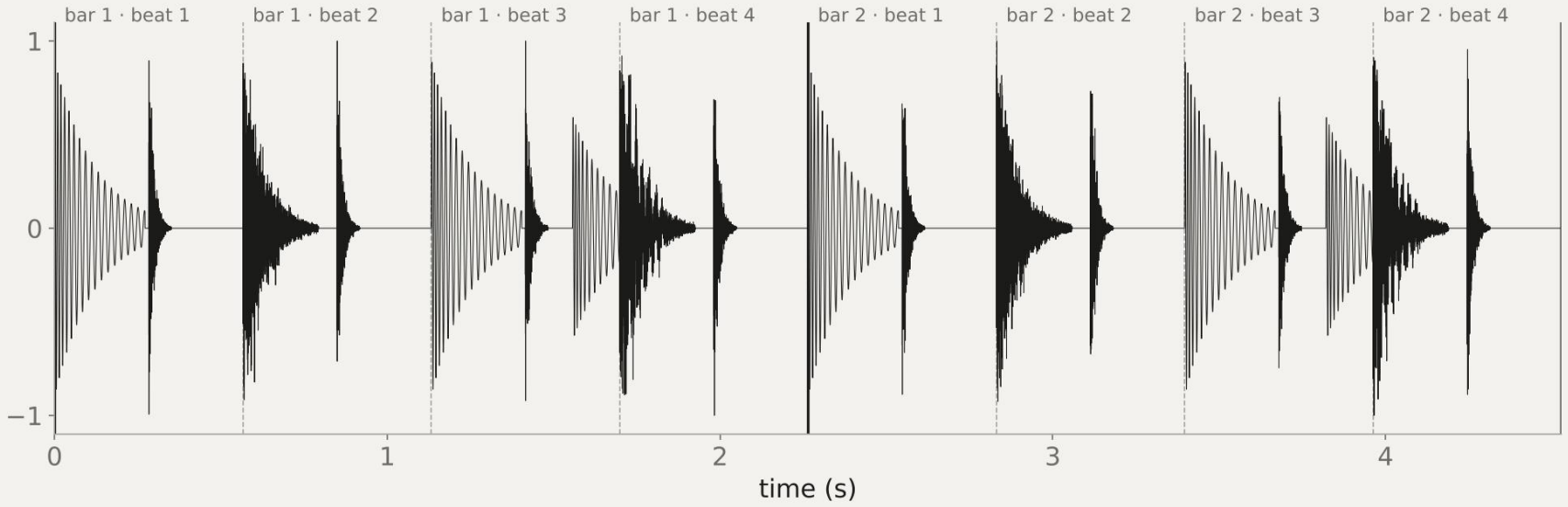
44100 samples = 2 s

…but seconds don't land on the beat



The cuts start and end in almost random places – not particularly **musical** ones.

A sidebar on musical timing



BAR

A larger musical segment, made of beats.

BEAT

The pulse of the music. Here: 4 per bar.

BPM

Beats per minute: sets how long each beat lasts. Here: **106**.

Working out samples per beat

```
samples_in_bar = len(audio) // 2
samples_in_beat = samples_in_bar // 4
```

99,849 // 2

49,924

samples in a bar

→

49,924 // 4

12,481

samples in a beat

≈

12,481 ÷ 22,050

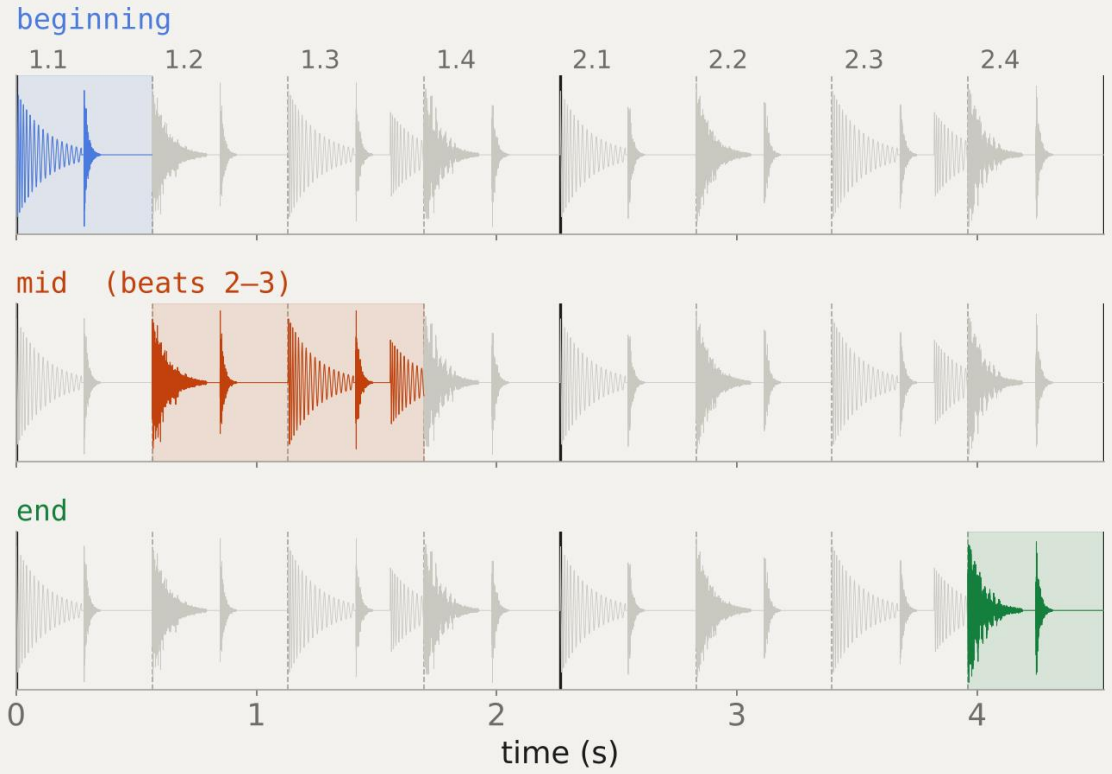
0.566 s

= 60 ÷ 106 bpm

// is **integer division**: it rounds down to a whole number – and an index has to be a whole number.

Cutting by beats

```
samples_in_bar = len(audio) // 2
samples_in_beat = samples_in_bar // 4
#This is the first beat
beginning = audio[:samples_in_beat]
print(len(beginning))
#This is the last beat
end = audio[-samples_in_beat:]
print(len(end))
#This is the middle of bar 1,
#the second and third beats
mid = audio[samples_in_beat:
        samples_in_beat*3]
print(len(mid))
```



Part 5

Pasting and editing

Put the pieces somewhere new – and change them on the way.

Overwrite · mix · volume · fades · reverse · speed

To paste, we need two things

1 Another **list** to copy it into

2 A **location** (in samples) of where to paste

Overwriting **one item**, last week:

```
colour[1] = 34
```

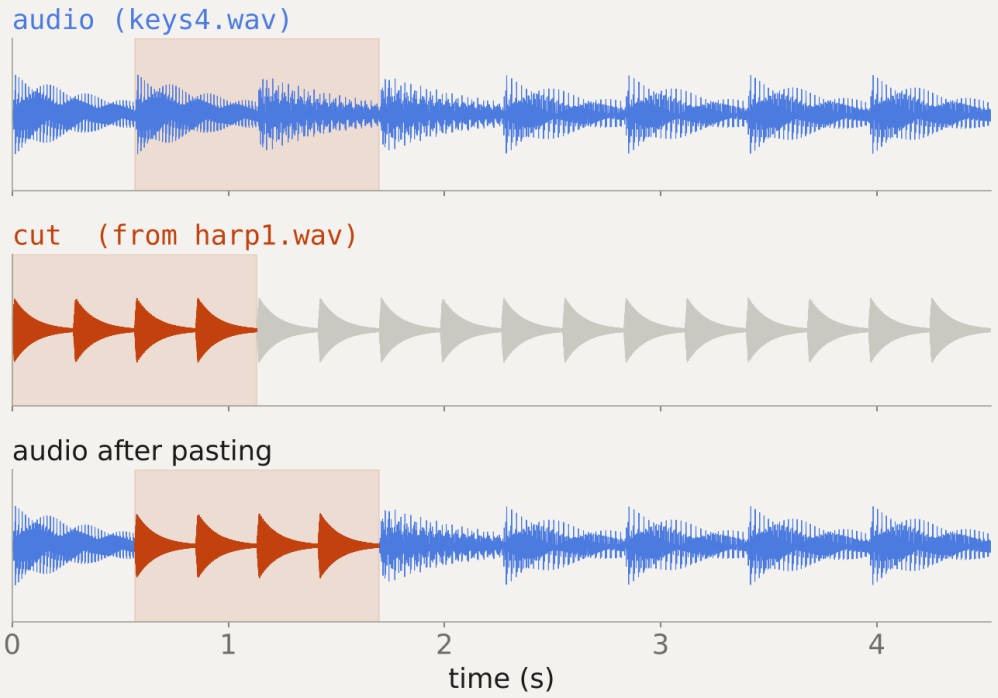


Overwriting a **whole section**:

```
audio[start:end] = cut
```

Overwriting

```
#week4-paste.py
#Load audio
audio, sr = librosa.load("audio/keys4.wav")
audio2, sr = librosa.load("audio/harp1.wav")
#How many samples?
samples_in_bar = len(audio) // 2
samples_in_beat = samples_in_bar // 4
#Cut
cut = audio2[:samples_in_beat*2 ]
#Overwrite
audio[samples_in_beat:samples_in_beat*3] = cut
```



All of the old audio in that range is **removed** and the new audio is put in its place.

The cut must be the same size as the gap

THIS WORKS

audio[12481:37443] = cut

gap: 24,962 samples · cut: 24,962 samples

THIS DOESN'T

audio[12481:24962] = cut

gap: 12,481 samples · cut: 24,962 samples

```
ValueError: could not broadcast input array from shape (24962,) into shape (12481,)
```

TIP Work out the end from the cut: `end = start + len(cut)`

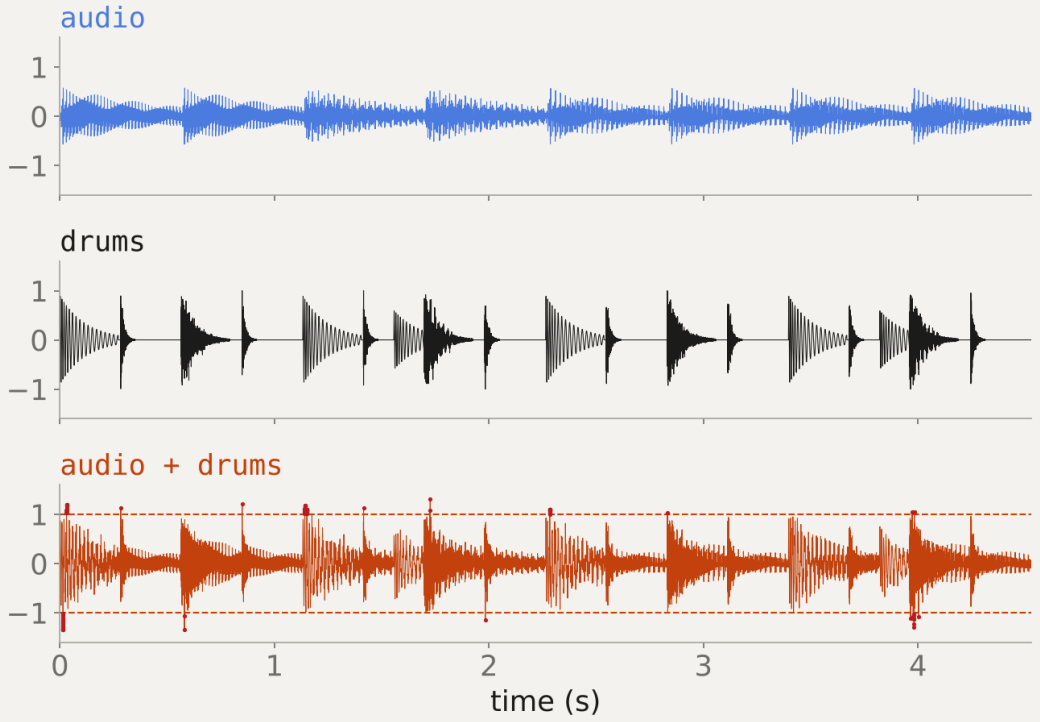
Mixing: add them together

```
#week4-mix.py
#Load audio
dot.music.start_sample_stream(audio + drums, sr=sr)
```

Item-wise addition: first + first, second + second…

```
[0.1, 0.5, -0.2]
+ [0.3, -0.1, 0.4]
= [0.4, 0.4, 0.2]
```

Works with **–**, ***** and **/** too – as long as the lists are **the same length**.



Lists join, arrays add

PYTHON LISTS

```
[1, 2] + [3, 4]
# [1, 2, 3, 4]
```

Joins them end to end: one after the other.

NUMPY ARRAYS (librosa)

```
np.array([1, 2]) + np.array([3, 4])
# array([4, 6])
```

Adds them item by item: on top of each other.

WANT TO JOIN ARRAYS? One after the other: `np.concatenate([audio, drums])`

Careful: louder than 1 distorts

THE LIMIT

Samples run from **−1** to **1**. Anything past that gets **clipped** – flattened – and sounds harsh.

IN THIS MIX

keys + drums peaks at about **1.35**: over a hundred samples go past the limit.

THE FIX

Turn each part down first: (audio + drums) * 0.5

Two sounds that each use the full range will usually be too loud added together.

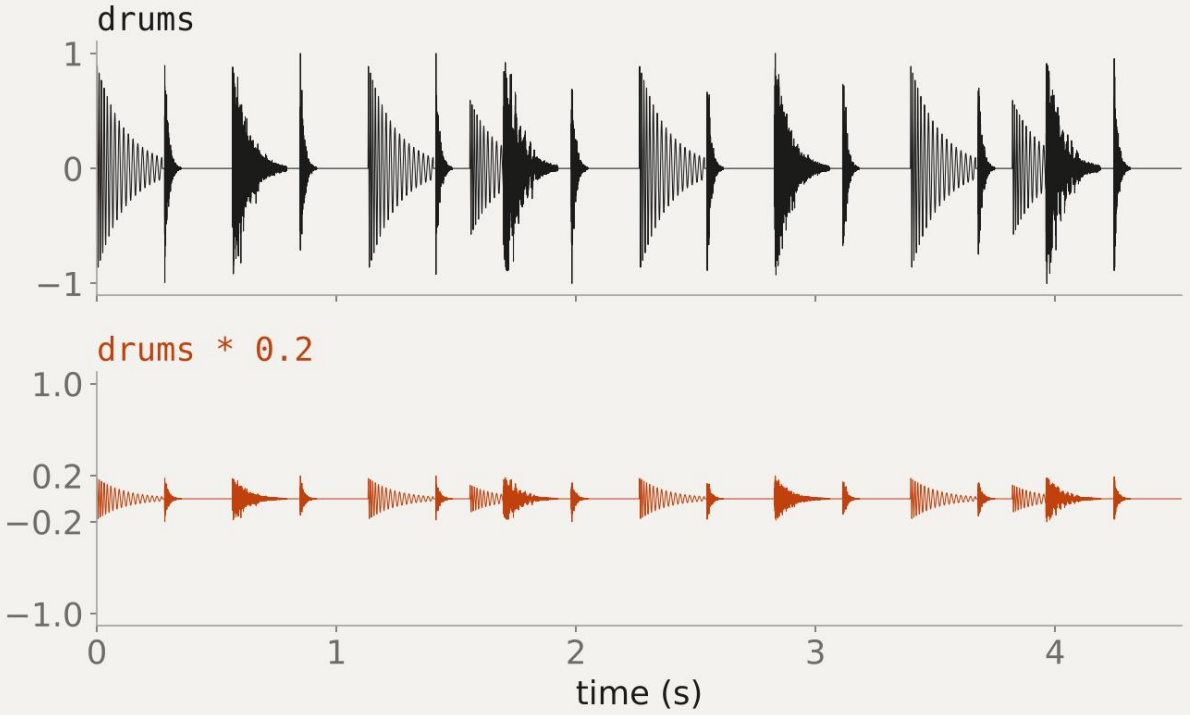
Volume: multiply

```
#week4-quieter.py
audio_output = drums * 0.2
```

Make all the numbers **smaller** (quieter) or **bigger** (louder).

Here every sample becomes **20%** of its original value.

More on how we perceive loudness next week.



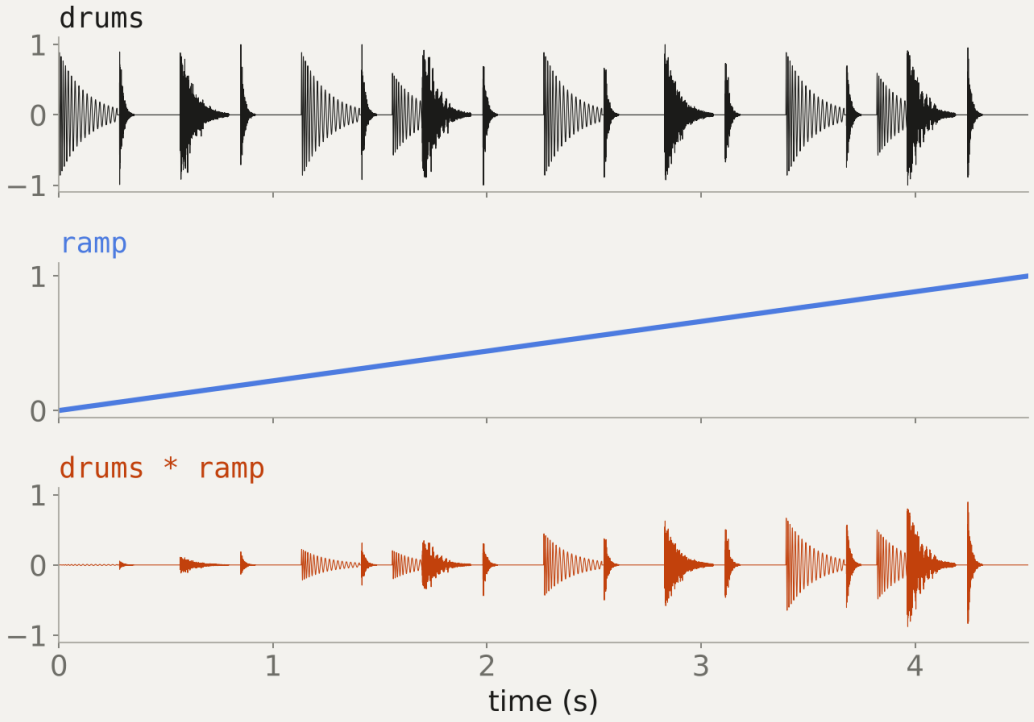
Fade in: multiply by a ramp

```
#week4-fadein.py
ramp = np.linspace(0,1,len(drums))
audio_output = drums * ramp
```

np.linspace(0, 1, n): n numbers going evenly from 0 to 1.

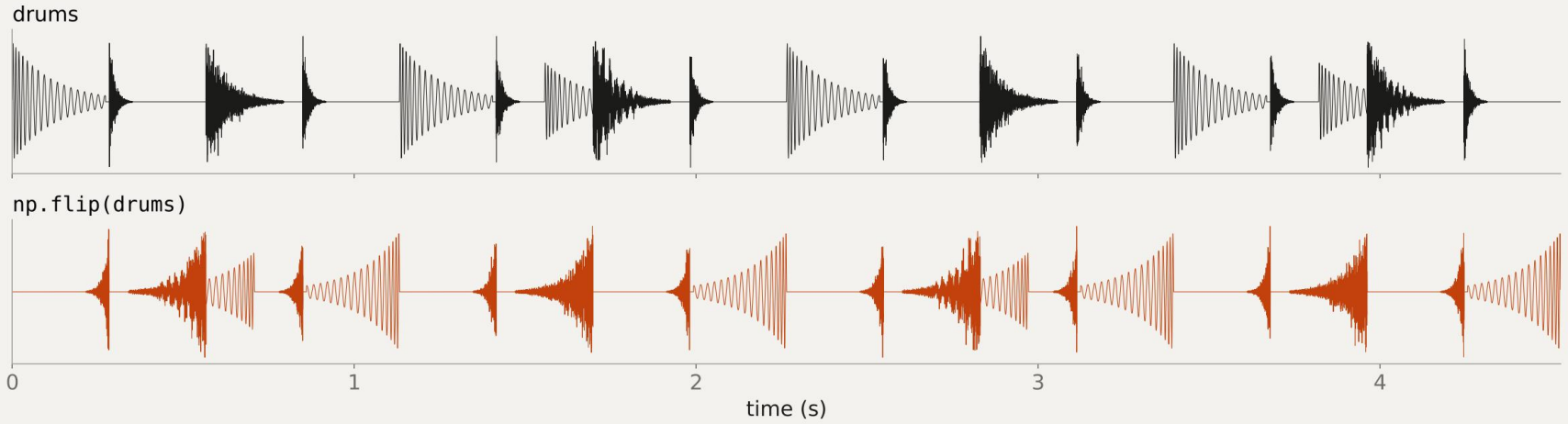
Each sample is multiplied by the matching ramp value: silent at the start, full volume at the end.

Fade out? np.linspace(1, 0, n)



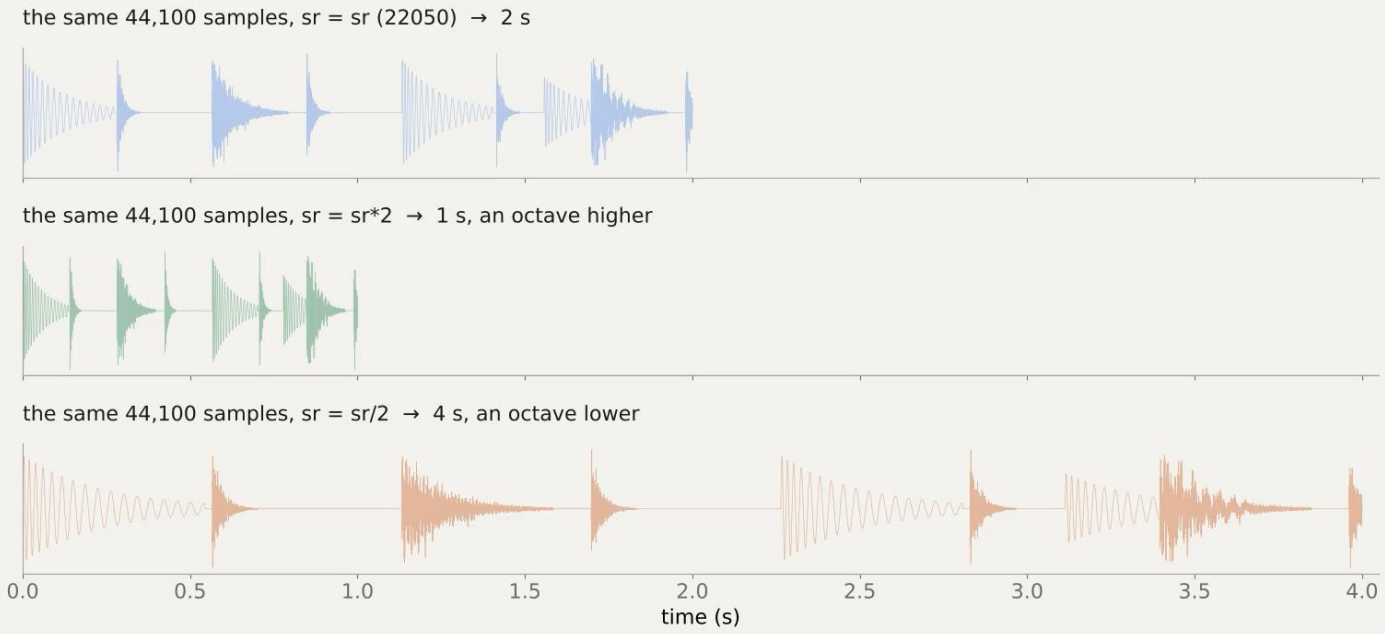
Reverse: flip the array

```
#week4-reverse.py
#Just reverse the array
audio_output = np.flip(drums)
```



The last sample becomes the first. Every hit now **swells in** instead of dying away.

Speed and pitch: change the sample rate



Same numbers, different amount of **time** per number. Click to hear it.

Faster and slower in code

```
#week4-faster.py
#We can play at a different speed
#(higher sample rate = faster playback)
#Also half the length
dot.music.start_sample_stream(
   audio_data, sr = sr*2)
```

```
#week4-slower.py
#We can play at a different speed
#(lower sample rate = slower playback)
#Also, twice the length
dot.music.start_sample_stream(
   audio_data, sr = sr/2)
```

THE SAME NUMBERS

At 44,100 Hz, 44,100 samples = **1 second**. At 22,050 Hz, they last **2 seconds**.

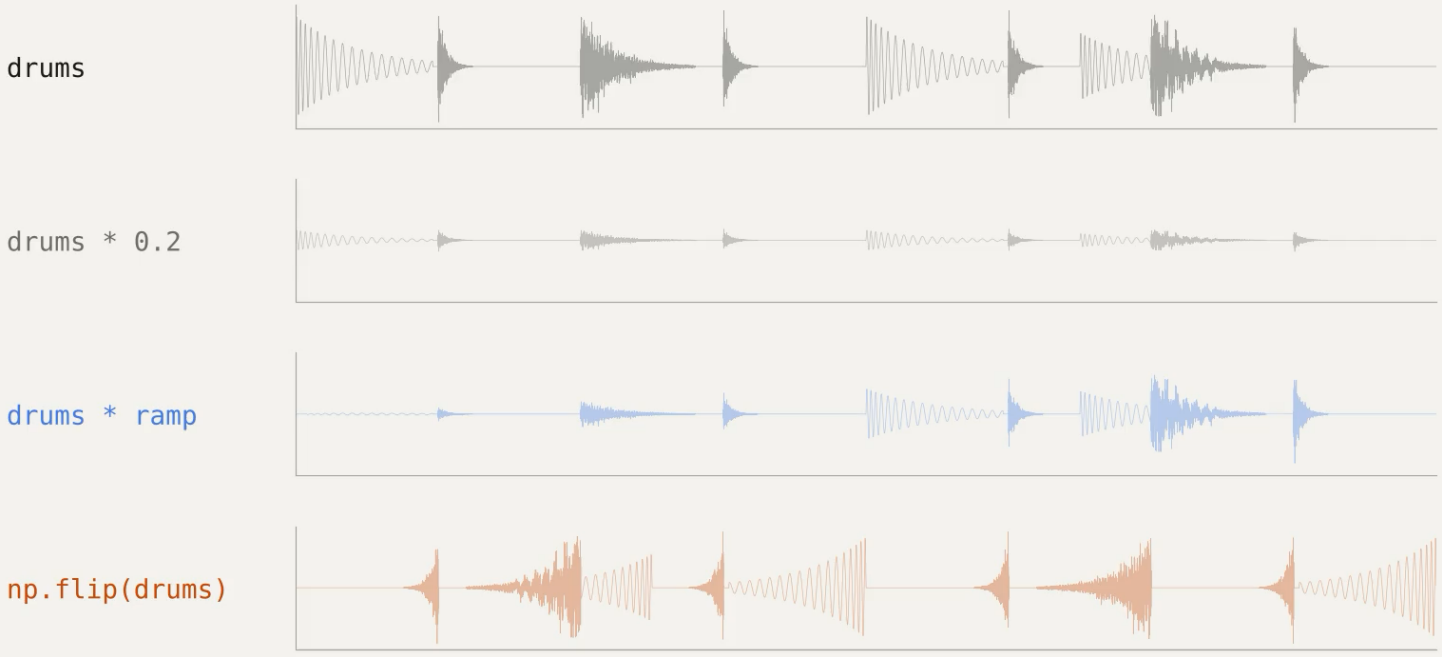
HIGHER RATE

Move through the samples quicker: **shorter** and **higher**.

LOWER RATE

Take longer to play through them: **longer** and **lower**.

Hear the edits



Click to play

● COLLAGE TASK

The third tape recorder

So far we've pasted into an **existing** audio file. But there's

NO REASON WE HAVE TO DO THIS.

Make a **blank canvas** – a third tape recorder – and paste into that.

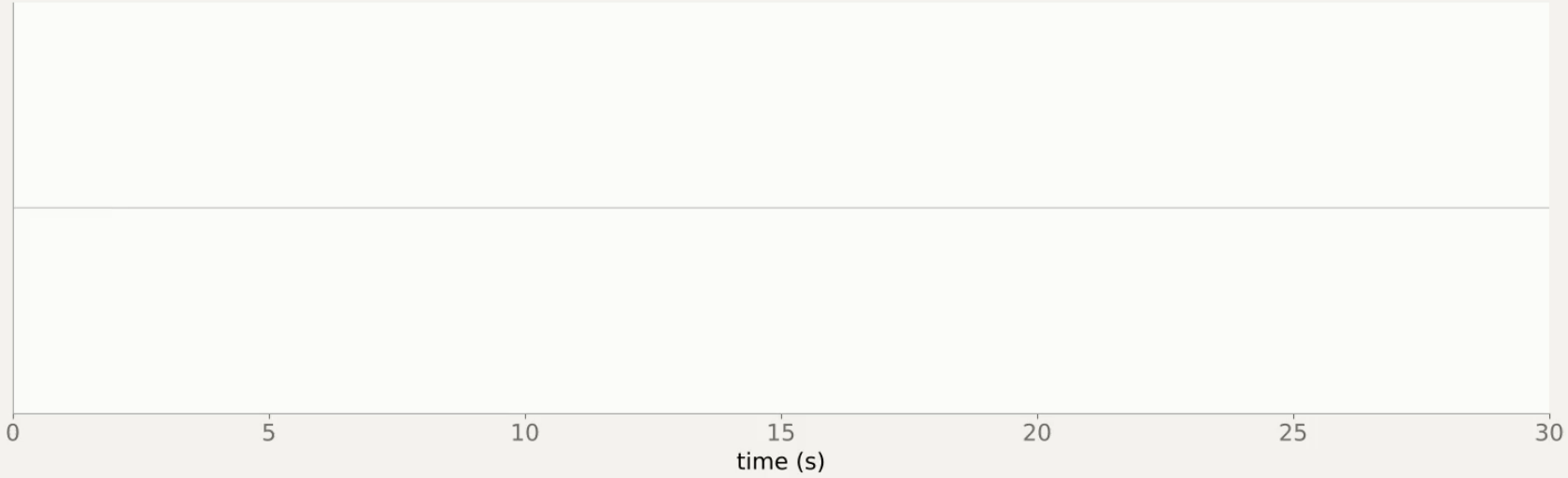
After Daphne Oram, *An Individual Note of Music*, p.62

```
audio, sr = librosa.load("audio/drums.wav")
#Make 60 of silence
blank_audio = np.zeros(sr*60)
#Define insert point in samples
start = 3*sr
#How long is our audio to paste in?
end = start + len(audio)
#Paste into blank track
blank_audio[start:end] = audio
```

COLLAGE TASK

Building a collage, one paste at a time

```
blank_audio[2*sr : 2*sr + len(clip)] = drums
```



Each clip: **cut** it, **edit** it, **paste** it at a start point. Then play the whole canvas.

● COLLAGE TASK

Your task: a sound collage

Source a variety of audio clips – music, environmental recordings, spoken word, found sounds – and use cut-and-paste operations to assemble them into a **new composition**.

VOLUME

Adjust clips to emphasise moments or create contrast.

CUT & REARRANGE

Rearrange or repeat subsections of a longer sample.

FADES

Control how sounds enter and exit – smooth or abrupt.

REVERSE

How does it change the texture and meaning of the sound?

SPEED

Slow down or speed up – what role does the clip play now?

Your final piece should reflect your **creative decisions** and the **editing techniques** you've applied.

COLLAGE TASK

Making it work

ABSTRACT OR MUSICAL?

Unexpected juxtapositions and textures – or a rhythmic, melodic sequence? Decide early.

OVERWRITE OR MIX?

Replace one sound with another, or layer them? Mixing: keep the total under ±1.

FIND GOOD CUT POINTS

Look at a sample in **Audacity** (free) to find where sounds start and stop – then note the time in seconds.

SECONDS → SAMPLES

Any time in seconds × sr = an index. Lengths must match: end = start + len(clip).

Audacity: audacityteam.org/download

SUMMARY

Four questions, answered

1 · WHAT IS SOUND – AND HOW IS IT RECORDED?

Vibrations in air. A microphone turns them into **voltage**; an ADC measures it thousands of times a second.

2 · WHAT IS INSIDE AN AUDIO FILE?

A **list of numbers** between −1 and 1. Samples ÷ sample rate = seconds.

3 · WHO CUT AND PASTED SOUND FIRST?

Composers with **tape**, DJs with **vinyl**, producers with **samplers** – from 1948 to today.

4 · HOW DO WE DO IT IN PYTHON?

Slice to cut, **assign** to paste, **+** to mix, ***** for volume and fades, np.flip, and the **sample rate** for speed.